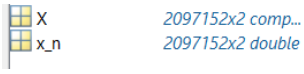
DSP Project

Student Names	
Abdelrhman Khaled Fouad	2100739
Mohamed Adel Abdelrahem	2100395
Moustafa Saad Dawood	2101506
Mohamed Hany Sarhan	2100452

content	page
Task 0	2-5
Task 1	6-21
Task 2	22-29
Task 3	30-37

Task 0 – reading an audio file

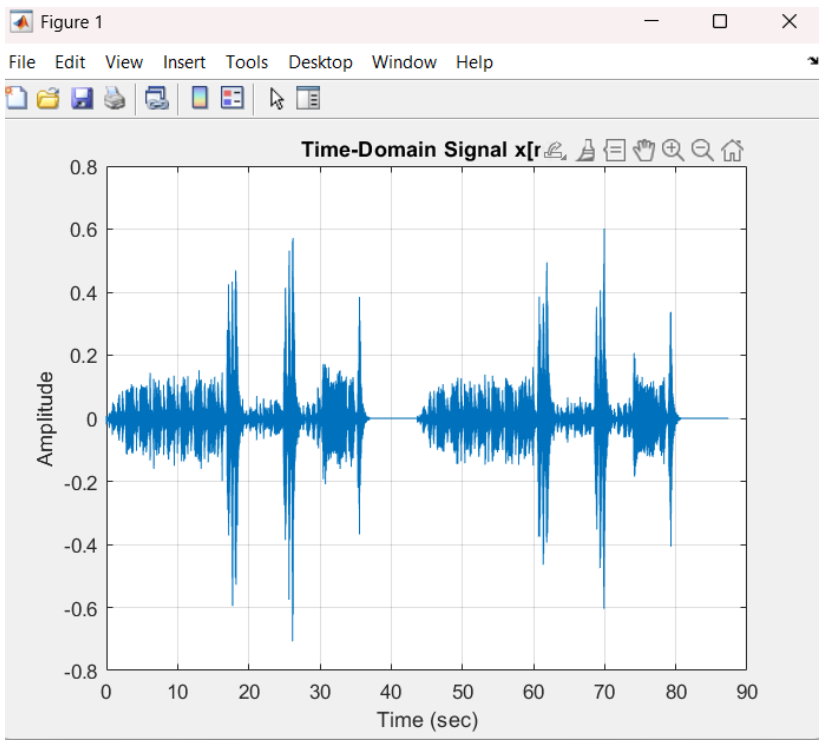
The normal x_n size is 

so it's not vector it is matrix, This means the audio is stereo (two channels). And MATLAB refuses to convolve a matrix with a vector. So we decided to force x_n into a vector by this line

```
x_n = x_n(:);  % force into column vector
```

this will make the audio look repeated in time domain.

Output wave form in time domain

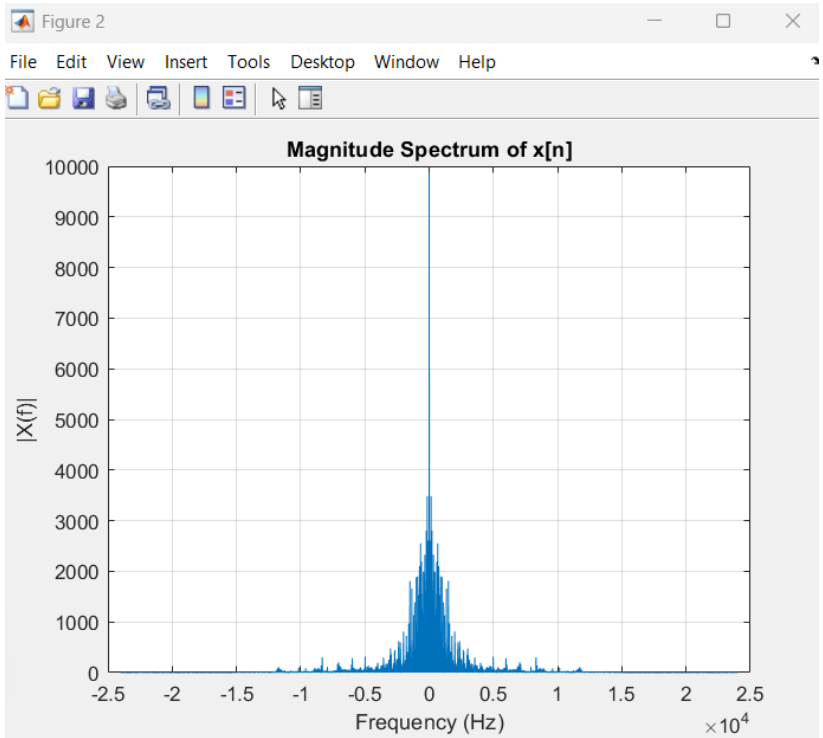


-The signal varies continuously with time, showing typical **speech/audio amplitude fluctuations**.

-Amplitude changes correspond to speech segments, pauses, and different loudness levels.

-The waveform confirms that the audio file was **loaded correctly and normalized naturally**.

Output in frequency domain



-The magnitude spectrum is **symmetric** around 0 Hz, which is expected because the audio signal is **real-valued**.

-Most energy is concentrated between **0 and ~8 kHz**, which is typical for speech and human voice.

-There is a strong peak at **0 Hz** (DC component).

This usually comes from:

mean offset in the signal or the nature of the FFT scaling and windowing.

Transcript window measuring Energy of signal and finding the Fs

```
Sampling frequency Fs = 48000.00 Hz

Total energy in time domain  = 5230.2114
Total energy in freq domain  = 5230.2114
Difference (should be very small) = 2.7285e-12
fx >>
```

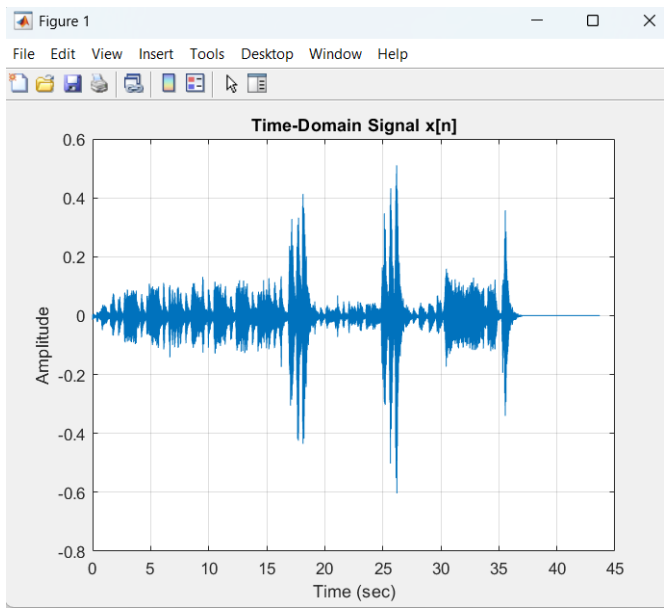
OR we can convert it into one mono signal by taking the average of 2 channels using this line

```
x_n = mean(x_n, 2);  % convert a stereo into mono signal so we can do conv of vector with vector
```

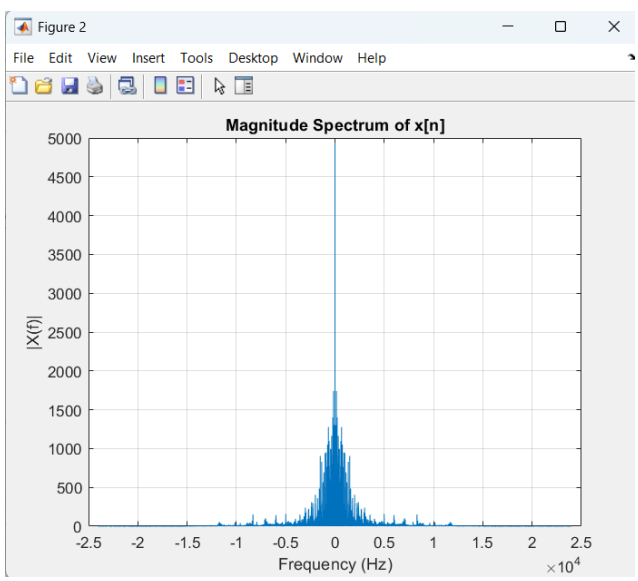
X	2097152x1 comp...
x_n	2097152x1 double

So the results will not be repeated

Output wave form in time domain



Output in frequency domain



```
Sampling frequency Fs = 48000.00 Hz

Total energy in time domain  = 2018.6258
Total energy in freq domain  = 2018.6258
Difference (should be very small) = 0.0000e+00
```

We will continue using this conversion stereo to mono so all results will be matching the final plots in task 0

Task 1 – Digital echo system

A digital echo system H can be represented by a general impulse response

$$h[n] = \sum_{k=0}^{\infty} b_k \delta[n - kD]$$

The added "echos" cause a distortion of the original signal. A digital equalizer system G can be used in order to remove this distortion. The impulse response of the equalizer is denoted by g[n], and it is related to h[n] as follows,

$$h[n] * g[n] = \delta[n]$$

• Let bk {1,0.9,0.8,0.7} and let D =1000.

• Using hand analysis, determine the LCCDE and the system function Hpzq of the digital echo system H.

Given

$b_k = \{1,\ 0.9,\ 0.8,\ 0.7\}$for $k = 0,1,2,3.$

$D = 1000.$

Impulse response (as given)

$$h[n] = \sum_{k=0}^{3} . b_k\ \delta[n - kD]$$

System function $\boldsymbol{H(z)}$

Take the z-transform of $h[n]$:

$$H(z) = \sum_{n=-\infty}^{\infty} . h[n] z^{-n} = \sum_{k=0}^{3} . b_k z^{-kD}.$$

So explicitly, $\boxed{H(z) = b_0 + b_1 z^{-D} + b_2 z^{-2D} + b_3 z^{-3D}}$

With the numeric b_k:

$$\boxed{H(z) = 1 + 0.9\, z^{-1000} + 0.8\, z^{-2000} + 0.7\, z^{-3000}}$$

LCCDE (input $x[n]$→ output $y[n]$)

Because the system is FIR (finite nonzero $h[n]$), the difference equation is simply convolution:

$$\boxed{y[n] = \sum_{k=0}^{3} . b_k\, x[n - kD]}$$

or numerically:

$$\boxed{y[n] = x[n] + 0.9\, x[n - 1000] + 0.8\, x[n - 2000] + 0.7\, x[n - 3000].}$$

• Use Matlab/GNU-Octave to generate the plots described in section 3 for the digital echo system system H.

3 General Requirements:

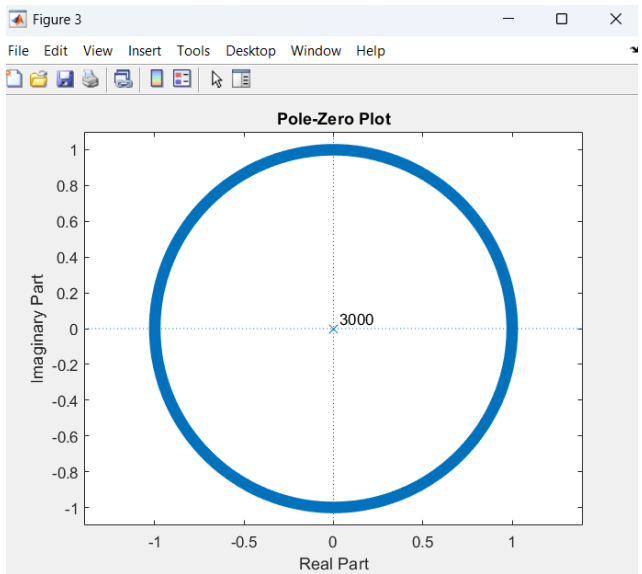
Assume that the sampling frequency used in your digital filter is F_s.
For all the filters in this assignment use Matlab / GNU-Octave to generate the following plots:

1. Pole-zero plot for the digital filter in the $\mathcal{Z}$−domain.
2. Magnitude and phase response of the filter in dB in the frequency range $(-F_s/2 < f \leqslant F_s/2)$.
3. Phase response of the filter in the frequency range $(-F_s/2 < f \leqslant F_s/2)$.
4. Group delay of the filter in the frequency range $(-\pi < \omega \leqslant \pi)$.
5. The filter's impulse response.

Please do NOT use Matlab GUI tools like filterDesigner or fvtool.

Figure (1)

There is only one pole at zero and there is 3000 roots so there is 3000 zeros are approximately on unit circle with different phase



zoomed

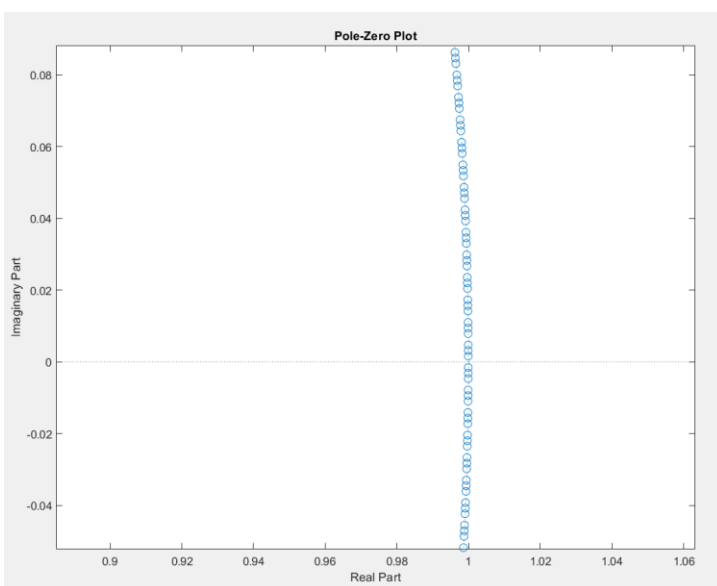


Figure (2) magnitude and phase response (-Fs/2 to Fs/2)

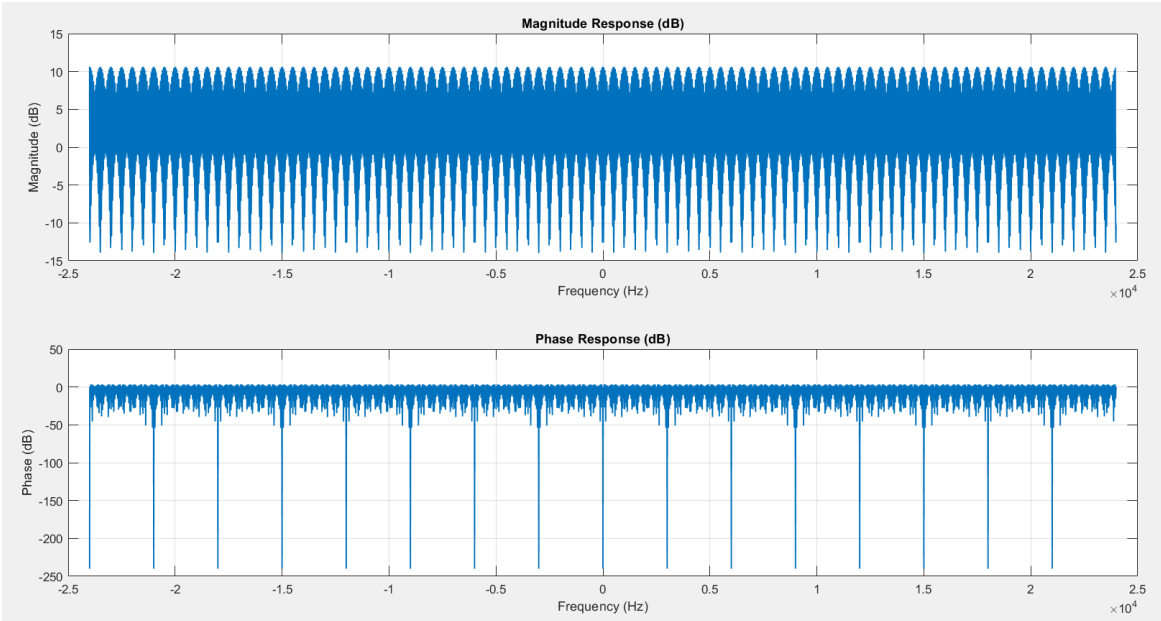


Figure (3) phase response in rad (-Fs/2 to Fs/2)

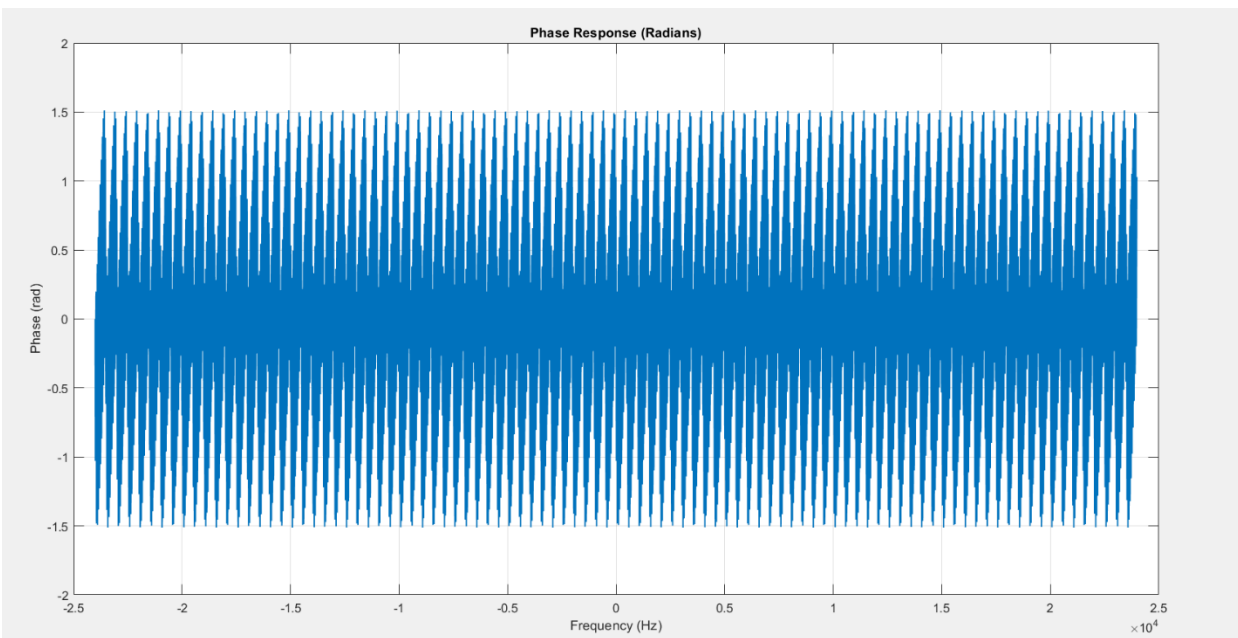


Figure (4) group delay (-pi to pi)

The group delay is the derivative of the phase

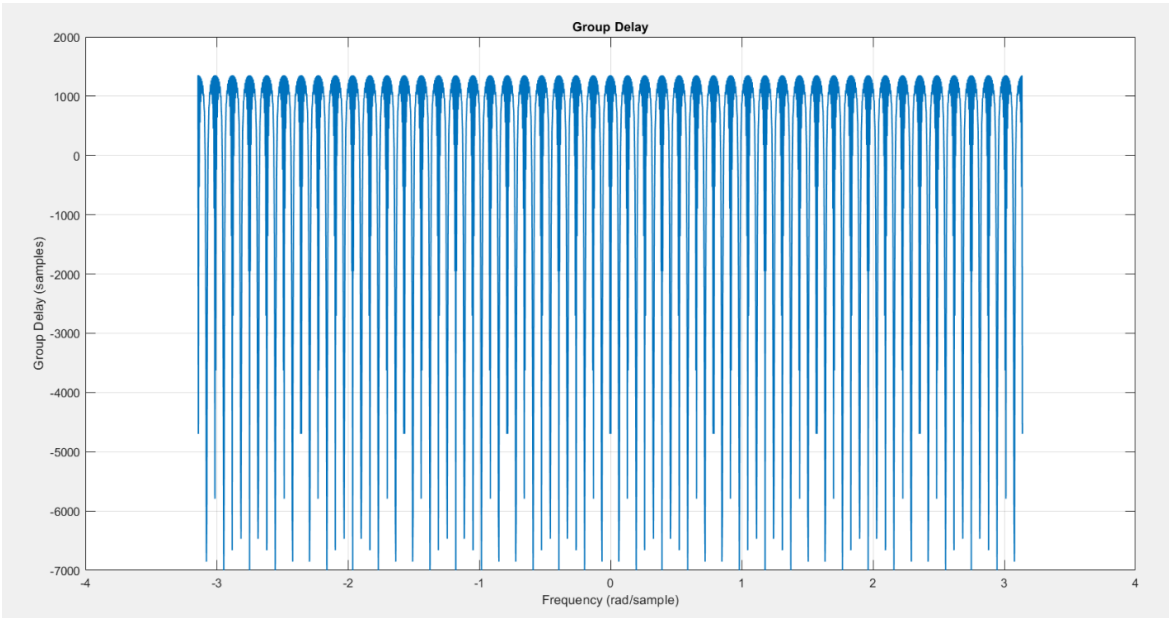
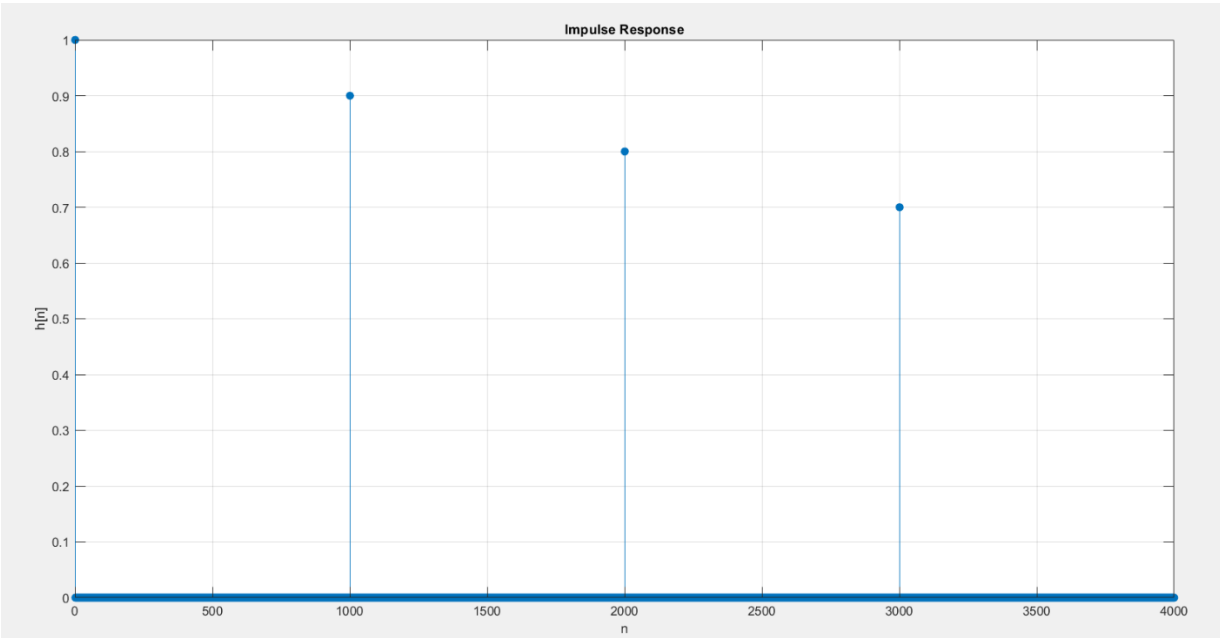
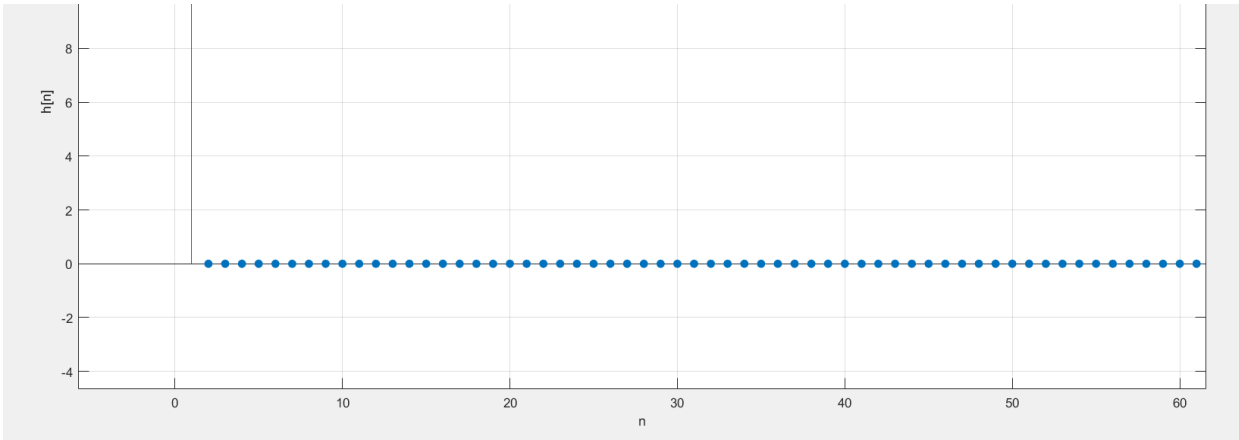


Figure (5) filter's impulse response

$$h[n]=\delta[n]+0.9\delta[n-1000]+0.8\delta[n-2000]+0.7\delta[n-3000]$$



zoomed



- Use the audio signal $x[n]$ as an input to this digital echo system $\mathcal{H}$, let's denote the output as $y_1[n]$.
- Compute the mean-square-error of $y_1[n]$ as follows:

$$\mathrm{MSE}_{y1} = \frac{1}{(N+1)} \sum_{n=0}^{n=N} (y_1[n] - x[n])^2$$

We used convolution (as it's said not to use filterDesigner tool in MATLAB) between x_n and h[n] where h[n] we get it in fig.5 we can say it's h[n]=b.

```
y_n = conv(x_n, b);   % convolution of input with impulse response h=b

%% Compute MSE
N_x = length(x_n);                    % length of original signal
% Compare only first N_x samples since convolution output is longer
MSEy1 = (1/(N_x+1)) * sum( (y_n(1:N_x) - x_n).^2 );

%% Display result
fprintf('Mean-Square Error (MSE) of y_n: %.6f\n', MSEy1);
```

```
Mean-Square Error (MSE) of y_n: 0.001888
```

- Using hand analysis, determine the LCCDE and the system function $G(z)$ of the equalizer system $\mathcal{G}$.

H(z)G(z)=1.

Invert $H(z)$:

$$\boxed{G(z) = \frac{1}{H(z)} = \frac{1}{1 + 0.9z^{-1000} + 0.8z^{-2000} + 0.7z^{-3000}}}$$

If $g[n]$is the impulse response of G, then the recursive LCCDE:

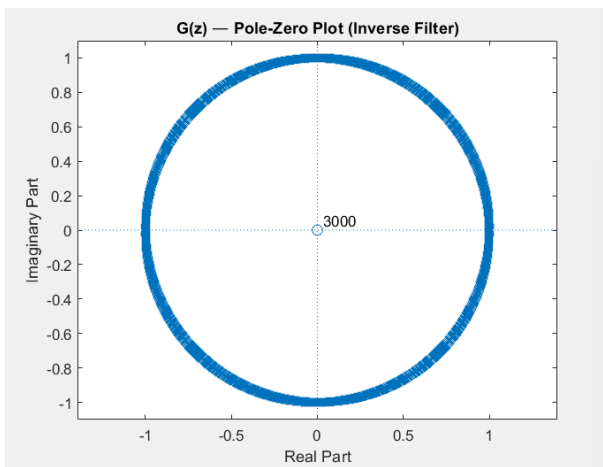
$$\boxed{g[n] = \delta[n] - 0.9\, g[n-1000] - 0.8\, g[n-2000] - 0.7\, g[n-3000]}$$

It's IIR filter , causal since it's depend only on past value not future.

- Use Matlab/GNU-Octave to generate the plots described in section 3 for the equalizer system $\mathcal{G}$.

Figure (1)

There is only one zero at zero and there is 3000 roots so there is 3000 poles are approximately on unit circle with different phase (inverse of H(Z)).



zoomed

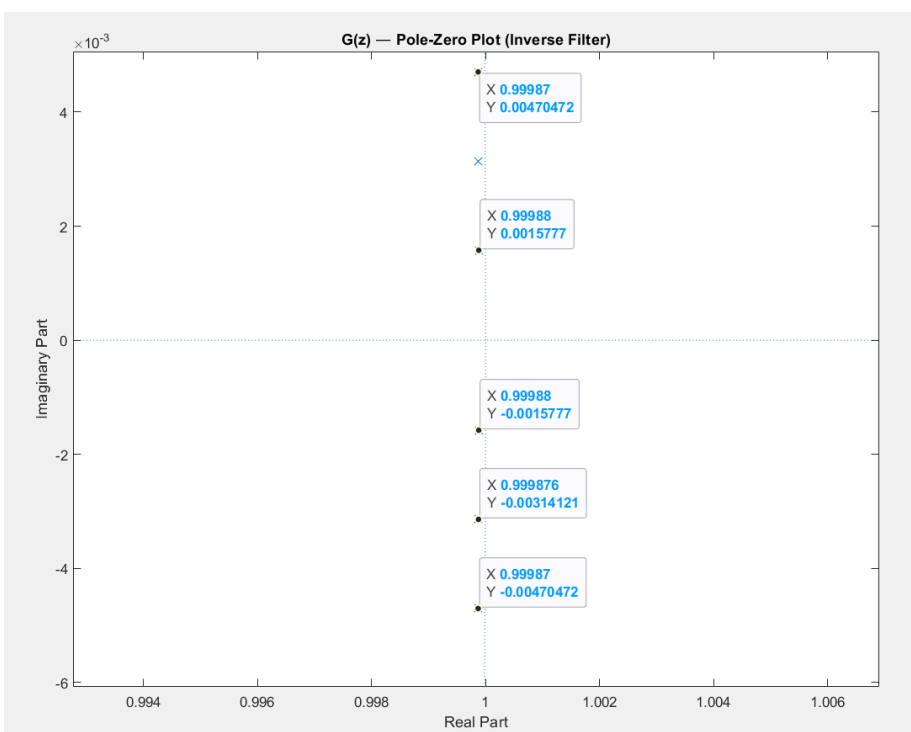


Figure (2) magnitude and phase response (-Fs/2 to Fs/2)

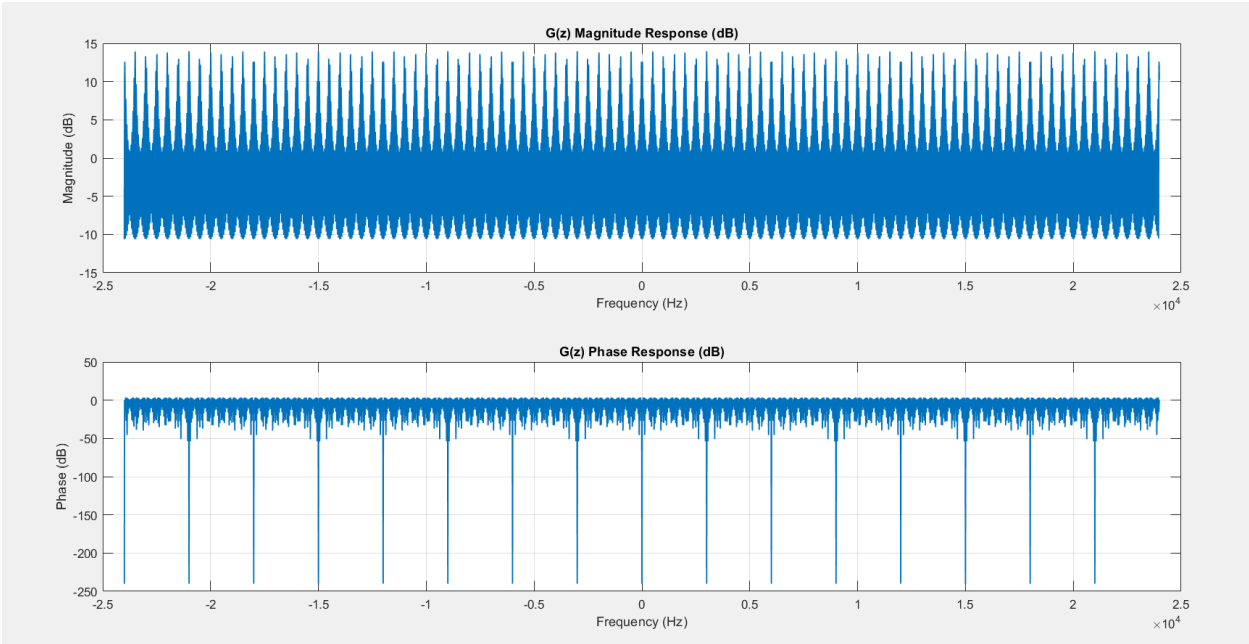
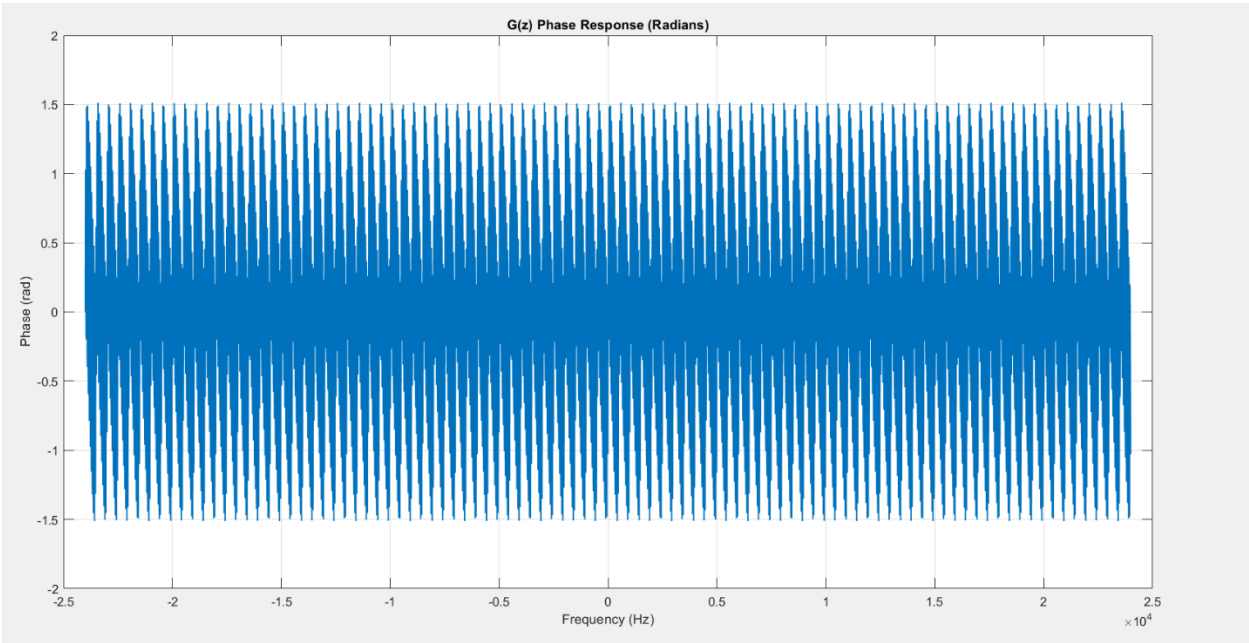


Figure (3) phase response in rad (-Fs/2 to Fs/2)



The curves are inverse of the curves of H.

Figure (4) group delay (-pi to pi)

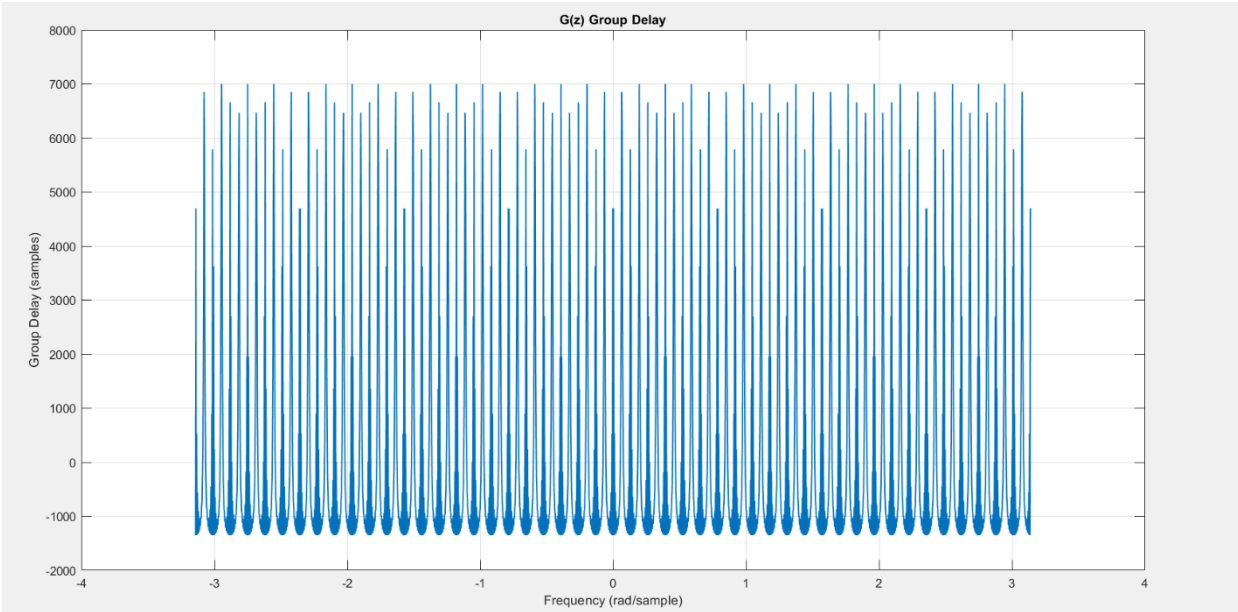
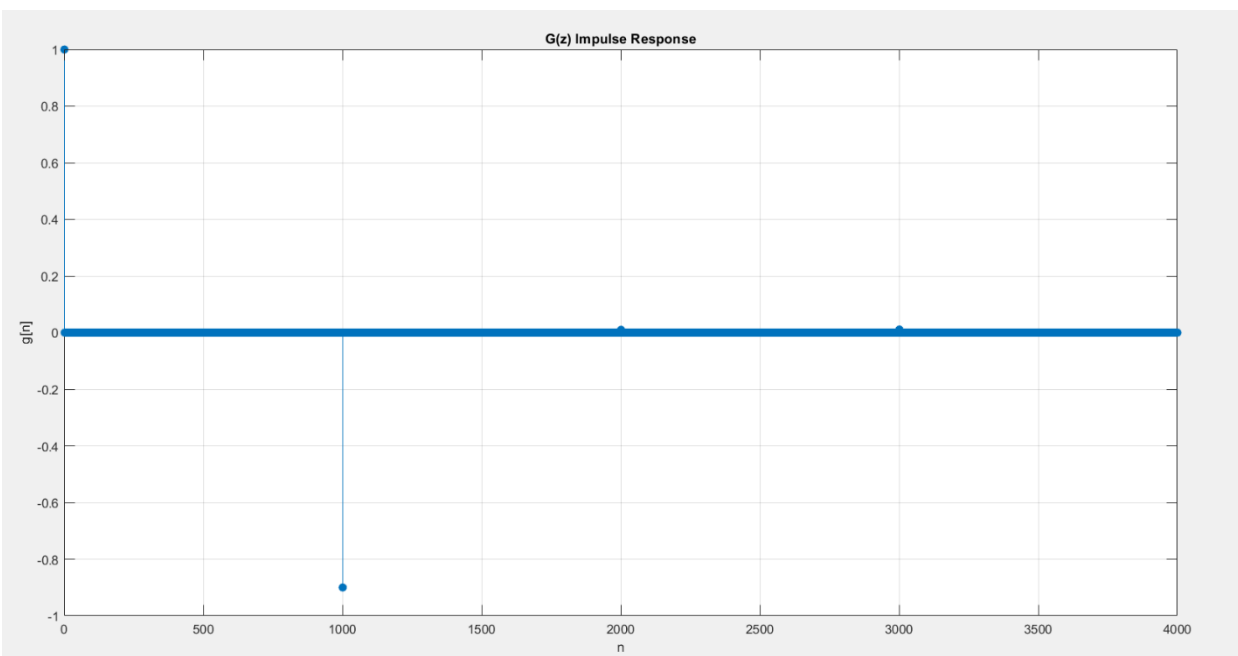
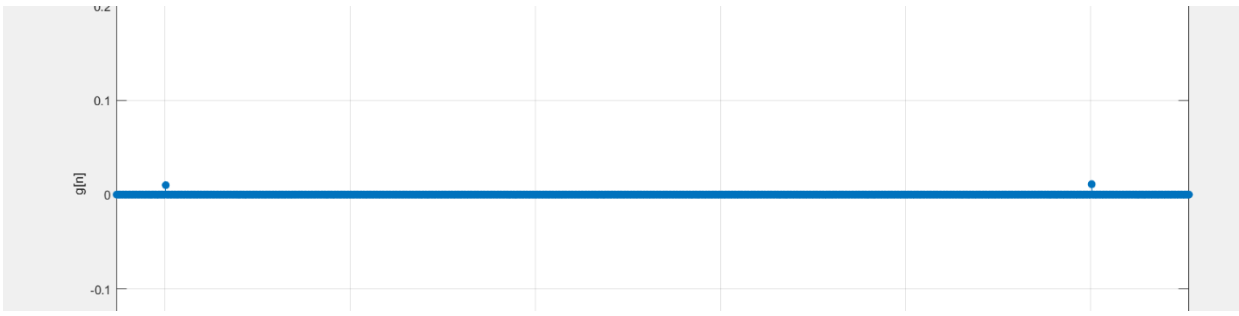


Figure (5) filter's impulse response

$$g[n]=\delta[n]-0.9g[n-1000]-0.8g[n-2000]-0.7g[n-3000]$$



zoomed



It should look like 4 impulse 1 at zero and -0.9 at 1000 and -0.8 at 2000 and -0.7 at 3000 but since it's feedback system so Because the system is IIR, those delayed terms feed back into each other.

Example:

$$g[2000] \text{ affects } g[3000], g[4000], g[5000], \ldots$$
$$g[3000] \text{ affects } g[4000], g[5000], g[6000], \ldots$$

so the impulses can grow or decay depending on the poles magnitude.

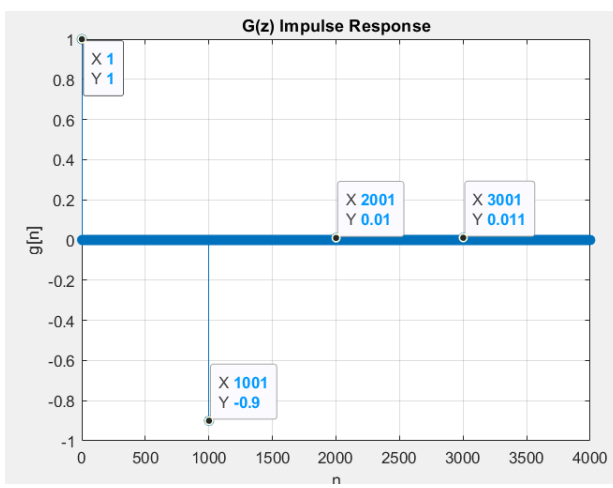
Let's calculate it

g[0]=1 ,

g[1000]=-0.9g[0]=-0.9

g[2000]=-0.9g[1000]-0.8g[0]=0.01

g[3000]=-0.9g[2000]-0.8g[1000]-0.7g[0]=0.011



- Apply the signal $y_1[n]$ (which has echos) to the equalizer system, and let's denote the output as $y_2[n]$.
- Compute the mean-square-error of $y_2[n]$ as follows:

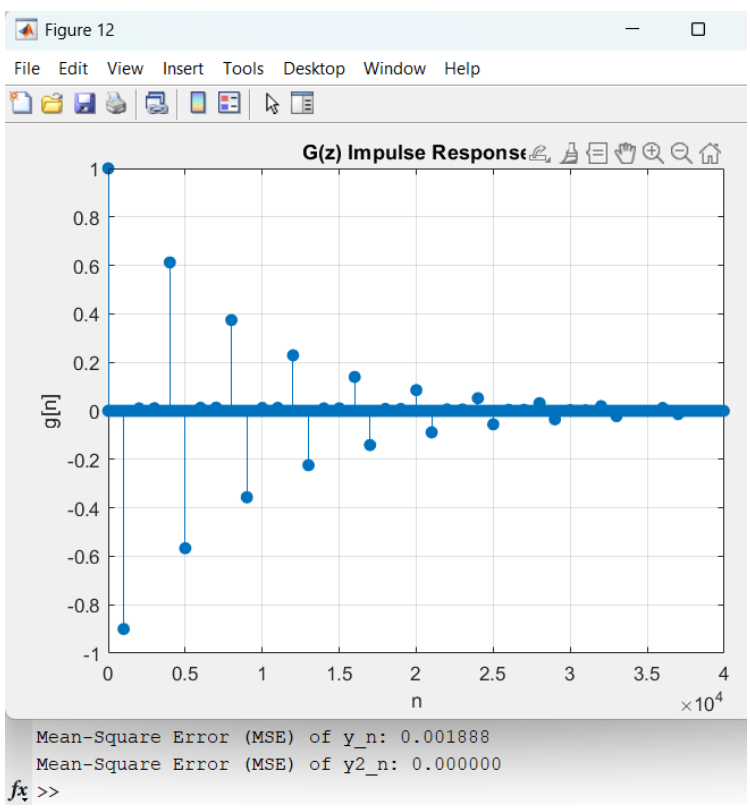
$$\mathrm{MSE}_{y2} = \frac{1}{(N+1)} \sum_{n=0}^{n=N} (y_2[n] - x[n])^2$$

We made convolution between y_n output of H and g_n so the output is y2_n it should look like the original signal x_n since we applied equalizer G=1/H.

And we calculated the MSE between the output y2_n and x_n it's expected to be lower than the MSE between y_n and x_n.

```
Mean-Square Error (MSE) of y_n: 0.001888
Mean-Square Error (MSE) of y2_n: 0.000361
fx >>
```

For impulse response of G(z) , g[n] is feedback so it continue to infinity so if we tried to plot more samples of impulse response the MSE will be lower.



Why it is decaying since the poles inside the unit circle.

CODES

TASK 0 and TASK 1

```
clc; clear; close all;
                                                    %%
                                %%%%%%%% Task 0 %%%%%%%%

%%% plot time and frequency domain of the Audio file and finding the energy from both
domains %%%%
%% 1. Read Audio File
[x_n, Fs] = audioread('sample_audio_file.wav');   % x: audio signal, Fs: sampling
frequency
x_n = mean(x_n, 2); % force into column vector (stereo to mono)

fprintf('Sampling frequency Fs = %.2f Hz\n', Fs);
N = length(x_n);
t = (0:N-1)/Fs;

%% 2. Plot time-domain signal x[n]
figure;
plot(t, x_n);
xlabel('Time (sec)'); ylabel('Amplitude');
title('Time-Domain Signal x[n]');
grid on;

%% 3. Plot magnitude spectrum |X(f)|
X = fftshift(fft(x_n));
f = (-N/2:N/2-1) * (Fs/N);

figure;
plot(f, abs(X));
xlabel('Frequency (Hz)'); ylabel('|X(f)|');
title('Magnitude Spectrum of x[n]');
grid on;

%% 4. Compute total energy (time domain)
E_time = sum(abs(x_n).^2);

%% 5. Compute total energy (frequency domain)
% Parseval: E = (1/N) * sum(|X[k]|^2)
E_freq = (1/N) * sum(abs(X).^2);

fprintf('\nTotal energy in time domain  = %.4f\n', E_time);
fprintf('Total energy in freq domain  = %.4f\n', E_freq);
fprintf('Difference (should be very small) = %.4e\n', abs(E_time - E_freq));
```

```
%                              %%% TASK 1 %%%
%                   % H(z)=1+0.9z^(-1000)+0.8z^(-2000)+0.7z^(-3000)
%% Define filter H(z)
Fs = 48000;
b = [1 zeros(1,999) 0.9 zeros(1,999) 0.8 zeros(1,999) 0.7];
a = 1;

%% Frequency response (whole range, then shift to -Fs/2 to Fs/2)
[H,w] = freqz(b,a,4096,'whole');        % w: 0 → 2π
f = (w - pi) * Fs / (2*pi);             % convert to -Fs/2 → Fs/2
H_shift = fftshift(H);                  % shift response

%% Magnitude in dB
mag_dB = 20*log10(abs(H_shift));

%% Phase in dB (requested)
phase = angle(H_shift);
phase_dB = 20*log10(abs(phase) + 1e-12);

%% ==========================================================
% 1. POLE-ZERO PLOT
%% ==========================================================
figure;
zplane(b,a);
title('Pole-Zero Plot');

%% ==========================================================
% 2. MAGNITUDE (dB) + PHASE (dB)  over (-Fs/2 , Fs/2)
%% ==========================================================
figure;

subplot(2,1,1);
plot(f, mag_dB, 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('Magnitude Response (dB)');
grid on;

subplot(2,1,2);
plot(f, phase_dB, 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Phase (dB)');
title('Phase Response (dB)');
grid on;

%% ==========================================================
% 3. PHASE RESPONSE ONLY (RADIANS) over (-Fs/2 , Fs/2)
%% ==========================================================
figure;
plot(f, unwrap(angle(H_shift)), 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Phase (rad)');
title('Phase Response (Radians)');
grid on;

%% ==========================================================
% 4. GROUP DELAY over (-π , π)
```

```
%% ==========================================================
[gd, w_gd] = grpdelay(b,a,2048,'whole');       % w_gd: 0 → 2π
w_gd_shift = w_gd - pi;                       % shift to -π → π

figure;
plot(w_gd_shift, gd, 'LineWidth', 1.1);
xlabel('Frequency (rad/sample)'); ylabel('Group Delay (samples)');
title('Group Delay');
grid on;

%% ==========================================================
% 5. IMPULSE RESPONSE
%% ==========================================================
N = 4000;
h = impz(b,a,N);

figure;
stem(h, 'filled');
xlabel('n'); ylabel('h[n]');
title('Impulse Response');
grid on;



%% ==========================================================
% Apply digital echo system using convolution
%% ==========================================================
% h(n) is already defined from impz(b,a,N) or you can use:
% h = b;  % FIR filter coefficients themselves are the impulse response

y_n = conv(x_n, b);   % convolution of input with impulse response h=b

%% Compute MSE
N_x = length(x_n);                  % length of original signal
% Compare only first N_x samples since convolution output is longer
MSEy1 = (1/(N_x+1)) * sum( (y_n(1:N_x) - x_n).^2 );

%% Display result
fprintf('Mean-Square Error (MSE) of y_n: %.6f\n', MSEy1);

%% ==========================================================
%                %%% TASK 1 — Equalizer G(z) = 1/H(z) %%%
% ==========================================================

% H(z) coefficients from previous section
bH = b;   % numerator of H(z)
aH = a;   % denominator (1)

%% Define G(z) = 1 / H(z)
bG = aH;      % numerator = 1
aG = bH;      % denominator = H(z) coefficients

%% Frequency response of G(z)
[HG, wG] = freqz(bG, aG, 4096, 'whole');    % wG from 0 to 2pi
fG = (wG - pi) * Fs / (2*pi);               % shift it from -Fs/2 to Fs/2
```

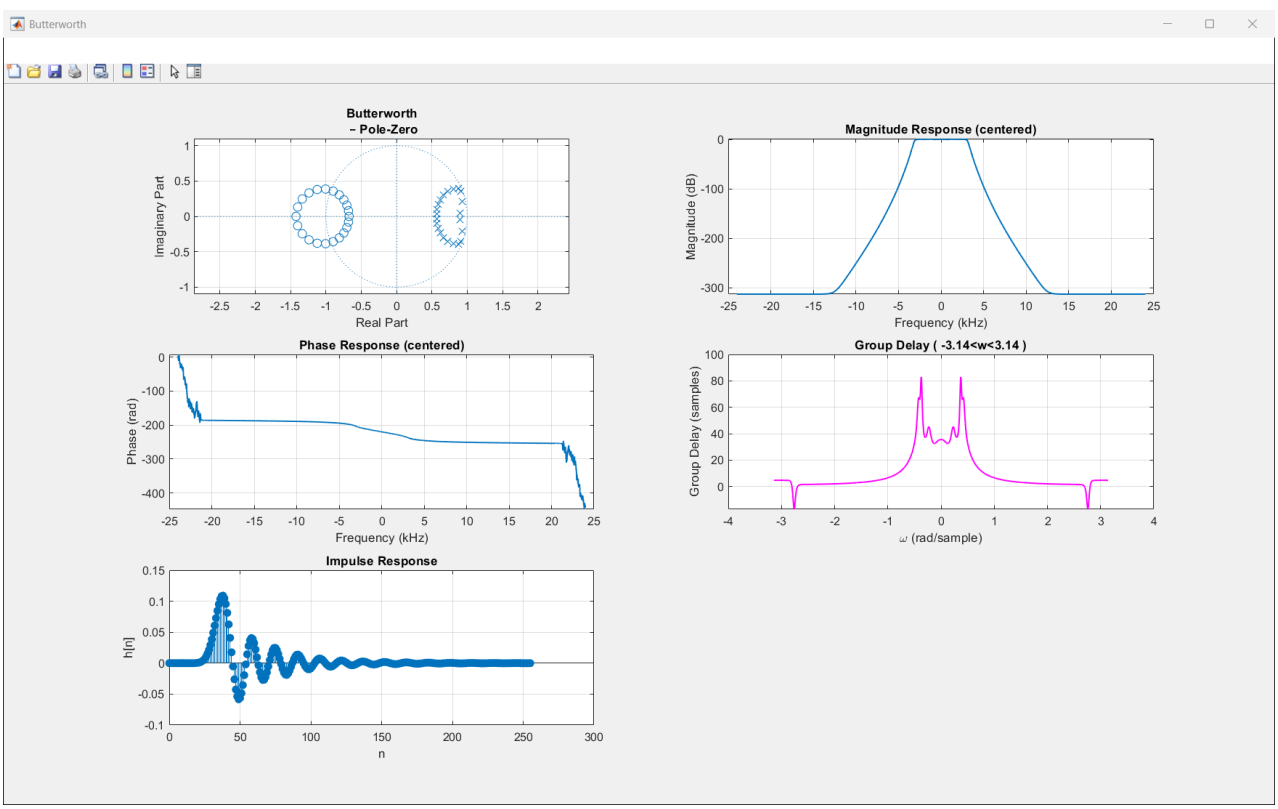
```
HG_shift = fftshift(HG);                    % shift response

%% Magnitude (dB)
magG_dB = 20*log10(abs(HG_shift));

%% Phase (dB)
phaseG = angle(HG_shift);
phaseG_dB = 20*log10(abs(phaseG) + 1e-12);

%% ==========================================================
% 1. POLE-ZERO PLOT of G(z)
%% ==========================================================
figure;
zplane(bG, aG);
title('G(z) — Pole-Zero Plot (Inverse Filter)');

%% ==========================================================
% 2. MAGNITUDE + PHASE in dB
%% ==========================================================
figure;

subplot(2,1,1);
plot(fG, magG_dB, 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Magnitude (dB)');
title('G(z) Magnitude Response (dB)');
grid on;

subplot(2,1,2);
plot(fG, phaseG_dB, 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Phase (dB)');
title('G(z) Phase Response (dB)');
grid on;

%% ==========================================================
% 3. PHASE RESPONSE (Radians)
%% ==========================================================
figure;
plot(fG, unwrap(angle(HG_shift)), 'LineWidth', 1.1);
xlabel('Frequency (Hz)'); ylabel('Phase (rad)');
title('G(z) Phase Response (Radians)');
grid on;

%% ==========================================================
% 4. GROUP DELAY
%% ==========================================================
[gdG, wGD] = grpdelay(bG, aG, 2048, 'whole');
wGD_shift = wGD - pi;

figure;
plot(wGD_shift, gdG, 'LineWidth', 1.1);
xlabel('Frequency (rad/sample)'); ylabel('Group Delay (samples)');
title('G(z) Group Delay');
grid on;

%% ==========================================================
```

```
% 5. IMPULSE RESPONSE of G(z)
%% ==========================================================
NG = 4000;
g_n = impz(bG, aG, NG);

figure;
stem(g_n, 'filled');
xlabel('n'); ylabel('g[n]');
title('G(z) Impulse Response');
grid on;


%% Apply equalizer system G to the echoed signal y_n
y2_n = conv(y_n, g_n);      % Output of the equalizer

%% Compute MSE between y2_n and original x_n
% Make the lengths match (use the first N_x samples)
MSEy2 = (1/(N_x+1)) * sum( (y2_n(1:N_x) - x_n).^2 );

%% Display result
fprintf('Mean-Square Error (MSE) of y2_n: %.6f\n', MSEy2);
```

Task 2 – Design of a Digital LPF

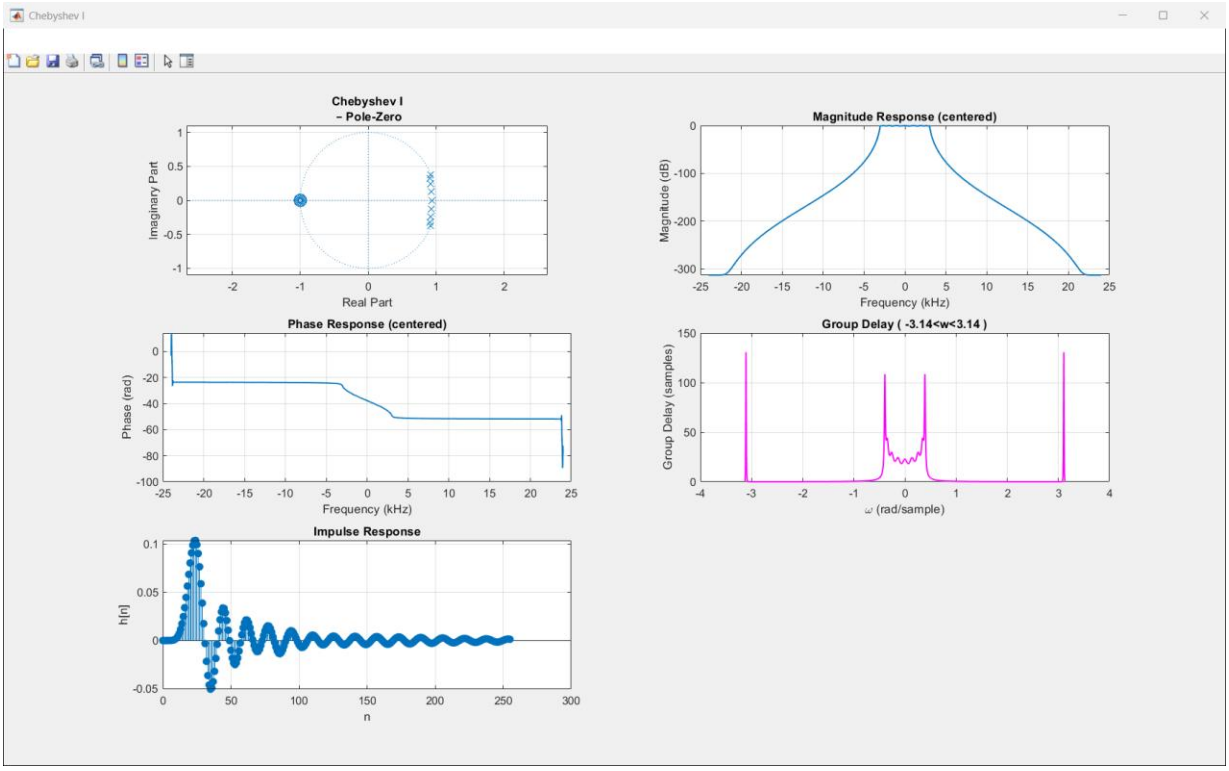
- Design a minimum order digital IIR LPF with the following specifications: $f_p = 3$ KHz, $f_s = 4$ KHz, $A_p \leqslant 1$ dB, and $A_s \geqslant 50$ dB.
- Repeat your design using the four filter types: Butterworth, Chebyshev Type I, Chebyshev Type II, and the Elliptic filters.
- Compare the filter order for the four types.
- Use the audio signal $x[n]$ as an input to each of the four filters.
- For each one of the four filters generate the plots described in section 3
- For each one of the four filters, compute the mean-square error between the input and output signals.
- For each one of the four filters, compute the percentage of energy lost due to the filtering process.
- Based on your results, which one of the four filters is better in terms of the minimum mean-square error between the input and output signals? which filter type results in a minimum percentage of lost power?

Butterworth Filter plot

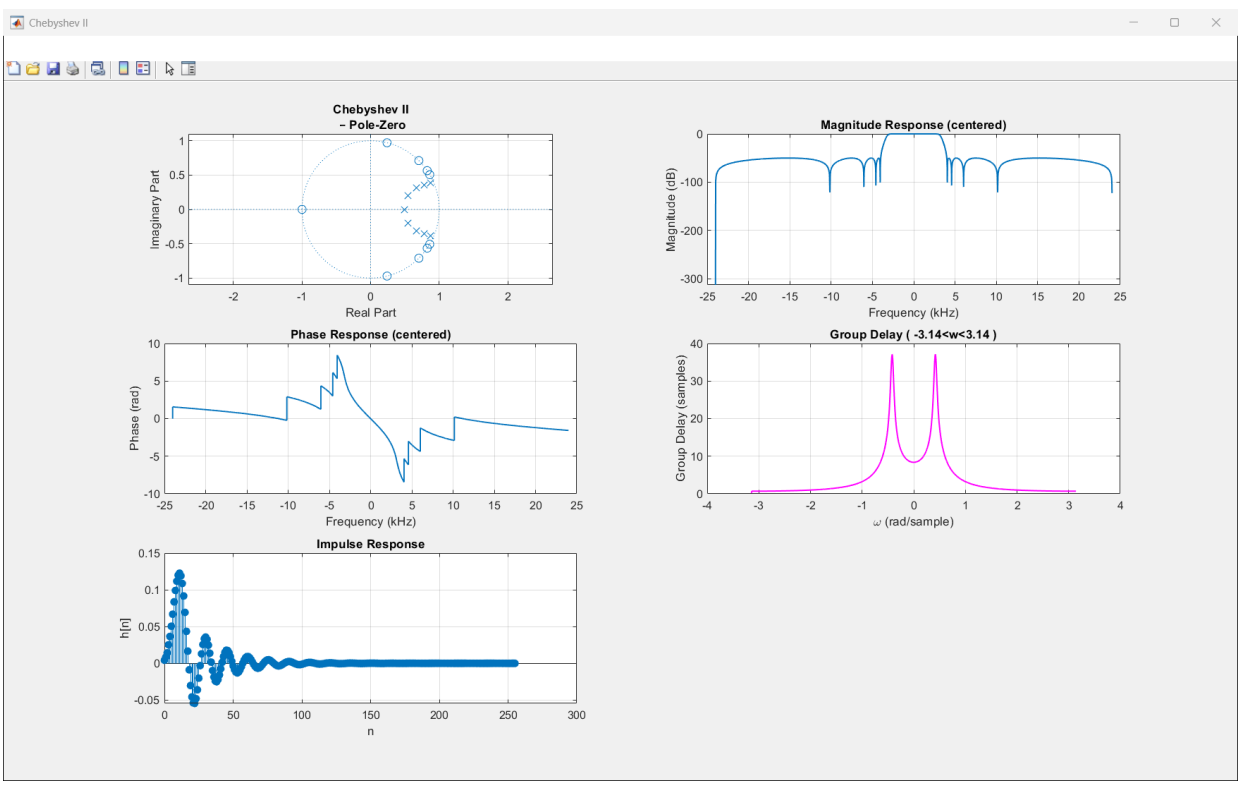


We plots the phase response using the command unwrap. And the magnitude response look as expected pass the 3k and start attenuate after that.

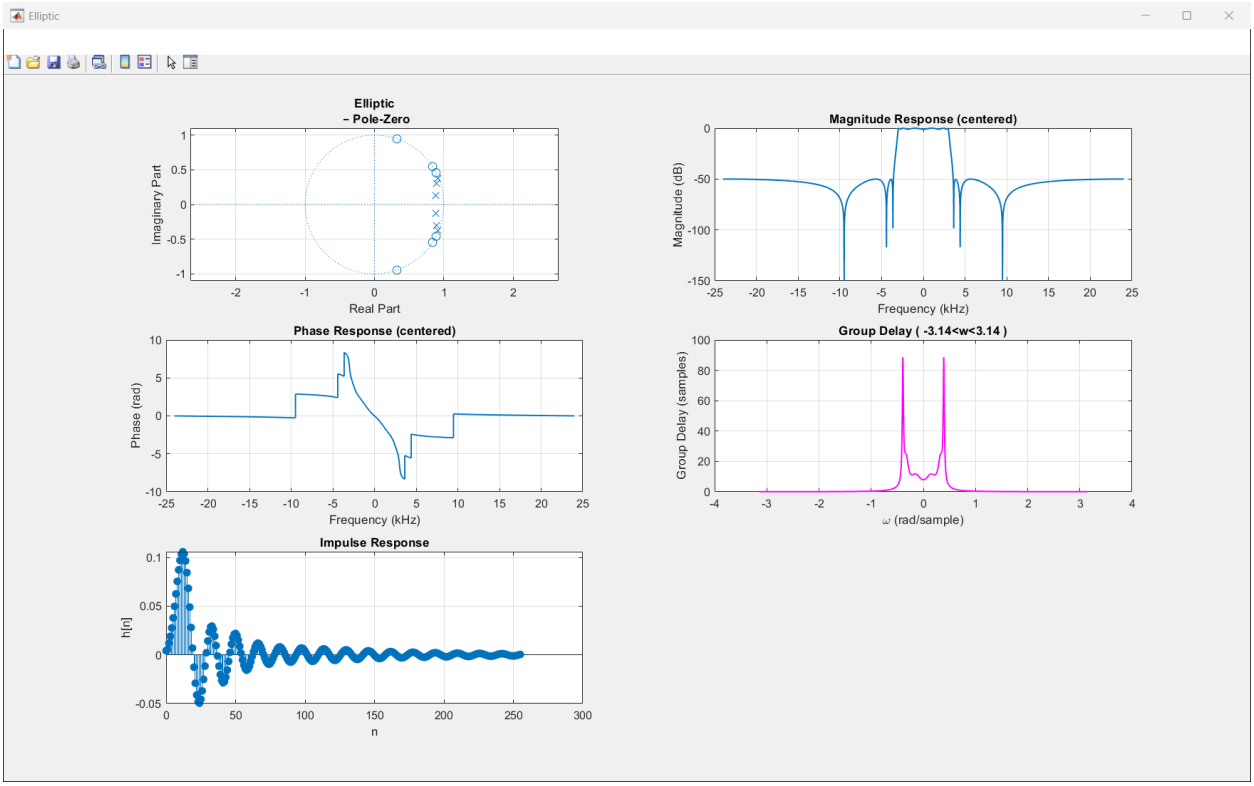
Chebyshev Type I plots



Chebyshev Type II plot



Elliptic plots



all pole and zero plots shows that all designed filters are stable since all poles lie inside unit circle , then the impulse response is decaying.

Butterworth filter has only poles and no finite zeros, which results in a smooth monotonic magnitude response.

The impulse responses decay with time, indicating stable IIR behavior.

Comparison between the 4 Types of filters for MSE and energy loss

```
Audio loaded: Fs = 48000 Hz

=== Butterworth Filter ===
Order = 22
MSE = 1.8358e-03 | Energy Lost = 2.8729 %

=== Chebyshev I Filter ===
Order = 9
MSE = 1.7637e-03 | Energy Lost = 15.5755 %

=== Chebyshev II Filter ===
Order = 9
MSE = 1.0205e-03 | Energy Lost = 3.1685 %

=== Elliptic Filter ===
Order = 6
MSE = 1.0718e-03 | Energy Lost = 14.1442 %

==================== SUMMARY ====================
Butterworth: Order=22 | MSE=1.836e-03 | Lost=2.87%
Chebyshev I: Order=9 | MSE=1.764e-03 | Lost=15.58%
Chebyshev II: Order=9 | MSE=1.021e-03 | Lost=3.17%
Elliptic: Order=6 | MSE=1.072e-03 | Lost=14.14%

Best (MSE): Chebyshev II
Best (Energy): Butterworth
==================================================
```

We conclude that the best filter for MSE is **Chebyshev II** with **MSE = 1.0205e-03**, the best filter for energy loss is **Butterworth** with **energy loss = 2.8729%** and the best filter to achieve the same specifications with **lowest order** is the **Elliptic filter**.

Filter	Order	MSE	Energy lost	Ripple location
Butterworth	22	1.83e-03	2.87% (best energy)	no ripple (maximally flat in passband)
Chebyshev I	9	1.76e-03	15.5% (worst)	ripples in passband (equiripple); stopband monotonic
Chebyshev II	9	1.02e-03 (best MSE)	3.16% (2nd best)	ripples in stopband (equiripple); passband monotonic
Elliptic	6	1.07e-03	14.14%	ripples in both passband and stopband (equiripple both sides)

Task 2 codes

```
%% ============================================================
%  Task 2 – Digital IIR LPF Design & Analysis (Centered Plots)
%  Filter types: Butterworth, Chebyshev I, Chebyshev II, Elliptic
%  All frequency plots from -Fs/2 to +Fs/2
% ============================================================

clear; clc; close all;

%% ------------------------------------------------------------
% Load audio
%% ------------------------------------------------------------
[x, Fs] = audioread("sample_audio_file.wav");
x = mean(x,2);  % averaging 2 channels like in task 1
%x = x(:);

fprintf("Audio loaded: Fs = %d Hz\n", Fs);

%% ------------------------------------------------------------
% Filter specifications
```

```
%% ------------------------------------------------------------
fp  = 3000;    % Passband edge
fsb = 4000;    % Stopband edge
Ap  = 1;       % Passband ripple (dB)
As  = 50;      % Stopband attenuation (dB)

Wp = fp/(Fs/2);
Ws = fsb/(Fs/2);

%% ------------------------------------------------------------
% Minimum-order filter designs
%% ------------------------------------------------------------
[Nb, Wnb]       = buttord(Wp, Ws, Ap, As);
[b_butt, a_butt] = butter(Nb, Wnb, "low");

[Nc1, Wnc1]     = cheb1ord(Wp, Ws, Ap, As);
[b_ch1, a_ch1]   = cheby1(Nc1, Ap, Wnc1, "low");

[Nc2, Wnc2]     = cheb2ord(Wp, Ws, Ap, As);
[b_ch2, a_ch2]   = cheby2(Nc2, As, Wnc2, "low");

[Ne, Wne]       = ellipord(Wp, Ws, Ap, As);
[b_ell, a_ell]   = ellip(Ne, Ap, As, Wne, "low");

%% ------------------------------------------------------------
% Filter the audio
%% ------------------------------------------------------------
y_butt = filter(b_butt, a_butt, x);
y_ch1  = filter(b_ch1 , a_ch1 , x);
y_ch2  = filter(b_ch2 , a_ch2 , x);
y_ell  = filter(b_ell , a_ell , x);

%% ------------------------------------------------------------
% Prepare filter info for looping
%% ------------------------------------------------------------
filters = {
    "Butterworth", b_butt, a_butt, y_butt, Nb;
    "Chebyshev I", b_ch1,  a_ch1,  y_ch1,  Nc1;
    "Chebyshev II",b_ch2,  a_ch2,  y_ch2,  Nc2;
    "Elliptic",    b_ell,  a_ell,  y_ell,  Ne
};

Nfft = 32768;         % FFT points for dense frequency response
results = struct();

%% ============================================================
% Loop over filters
%% ============================================================
for k = 1:size(filters,1)

    name  = filters{k,1};
    b     = filters{k,2};
    a     = filters{k,3};
    y     = filters{k,4};
    order = filters{k,5};
```

```
    fprintf("\n=== %s Filter ===\n", name);
    fprintf("Order = %d\n", order);

    %% ------------------- Frequency Response (centered) -------------------
    [H_whole, ~] = freqz(b,a,Nfft,'whole');  % 0..2π
    Hs = fftshift(H_whole);                  % center → -π..π

    f_center = (-Nfft/2:Nfft/2-1)*(Fs/Nfft); % -Fs/2 → Fs/2 (Hz)

    %% ------------------- Phase -------------------
    phase = angle(Hs);
    phase_un = unwrap(phase); % what unwrap do it plot every pi take a magnitude and
then connect the point together

    %% ------------------- Group Delay (centered -pi to pi) -------------------
    [gd, ~] = grpdelay(b,a,Nfft,'whole');  % 0..2π
    gd = fftshift(gd);                     % shift to -π..π

    %% ------------------- Impulse Response -------------------
    imp_len = 256;
    [h_imp, n_imp] = impz(b,a,imp_len);

    %% ------------------- Metrics -------------------
    mse = mean((x - y).^2);
    Ein = sum(x.^2);
    Eout = sum(y.^2);
    loss = (1 - Eout/Ein)*100;
    if loss < 0, loss = 0; end

    results(k).name = name;
    results(k).order = order;
    results(k).mse = mse;
    results(k).loss = loss;

    fprintf("MSE = %.4e | Energy Lost = %.4f %%\n", mse, loss);

    %% ==========================================================
    %     ONE FIGURE PER FILTER (all subplots inside)
    %% ==========================================================
    figure('Name',name,'NumberTitle','off','Position',[100 50 1400 900]);

    %----------------------------------------------------------
    % 1) Pole-Zero
    %----------------------------------------------------------
    subplot(3,2,1);
    zplane(b,a);
    title([name " – Pole-Zero"]);
    grid on;

    %----------------------------------------------------------
    % 2) Magnitude Response (centered)
    %----------------------------------------------------------
    subplot(3,2,2);
    plot(f_center/1e3, 20*log10(abs(Hs)+eps),'LineWidth',1.2);
```

```
    grid on;
    xlabel("Frequency (kHz)");
    ylabel("Magnitude (dB)");
    title("Magnitude Response (centered)");

    %-----------------------------------------------------------
    % 3) Phase Response (centered, unwrapped)
    %-----------------------------------------------------------
    subplot(3,2,3);
    plot(f_center/1e3, phase_un,'LineWidth',1.1);
    grid on;
    xlabel("Frequency (kHz)");
    ylabel("Phase (rad)");
    title("Phase Response (centered)");

    %-----------------------------------------------------------
    % 4) Group Delay (centered, -π ≤ w ≤ π)
    %-----------------------------------------------------------
    subplot(3,2,4);
    omega = linspace(-pi, pi, Nfft);   % rad/sample
    plot(omega, gd, 'm','LineWidth',1.2);
    grid on;
    xlabel("\omega (rad/sample)");
    ylabel("Group Delay (samples)");
    title("Group Delay ( -3.14<w<3.14 )");


    %-----------------------------------------------------------
    % 5) Impulse Response
    %-----------------------------------------------------------
    subplot(3,2,5);
    stem(n_imp, h_imp,'filled');
    grid on;
    xlabel("n");
    ylabel("h[n]");
    title("Impulse Response");

end

%% ============================================================
% PRINT SUMMARY
%% ============================================================
fprintf("\n==================== SUMMARY ====================\n");
for k = 1:4
    fprintf("%s: Order=%d | MSE=%.3e | Lost=%.2f%%\n", ...
        results(k).name, results(k).order, results(k).mse, results(k).loss);
end

[~,idx1] = min([results.mse]);
[~,idx2] = min([results.loss]);

fprintf("\nBest (MSE): %s\n", results(idx1).name);
fprintf("Best (Energy): %s\n", results(idx2).name);
fprintf("==================================================\n\n");
```

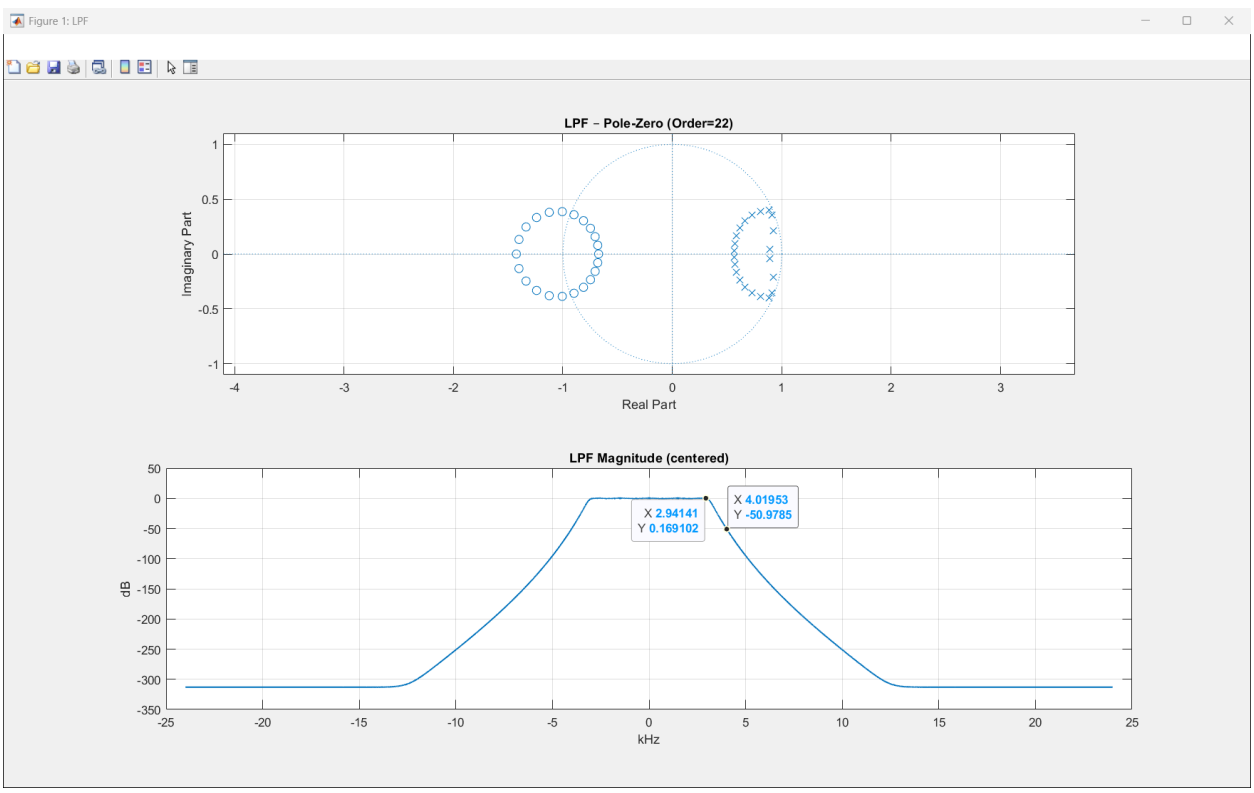
Task 3 – Transformation of LPF to HPF and BPF

7 Task 3: Frequency Transformation using Pole-Zero Pattern Rotation

Rotating the pole-zero pattern can be used to transform a lowpass filter into a highpass or a bandpass filter. This is based on the frequency shift property of the DTFT.

- Transform the Butterworth LPF you have designed in section 6 into a highpass filter by rotating the pole-zero pattern by π. Generate the plots described in section 3 for your HPF.
- Employ the same concept to transform the Butterworth LPF in section 6 into a bandpass filter centered around $\pi/2$. What would you do to make sure that the filter coefficients are real valued? Generate the plots described in section 3 for your BPF.

The LPF

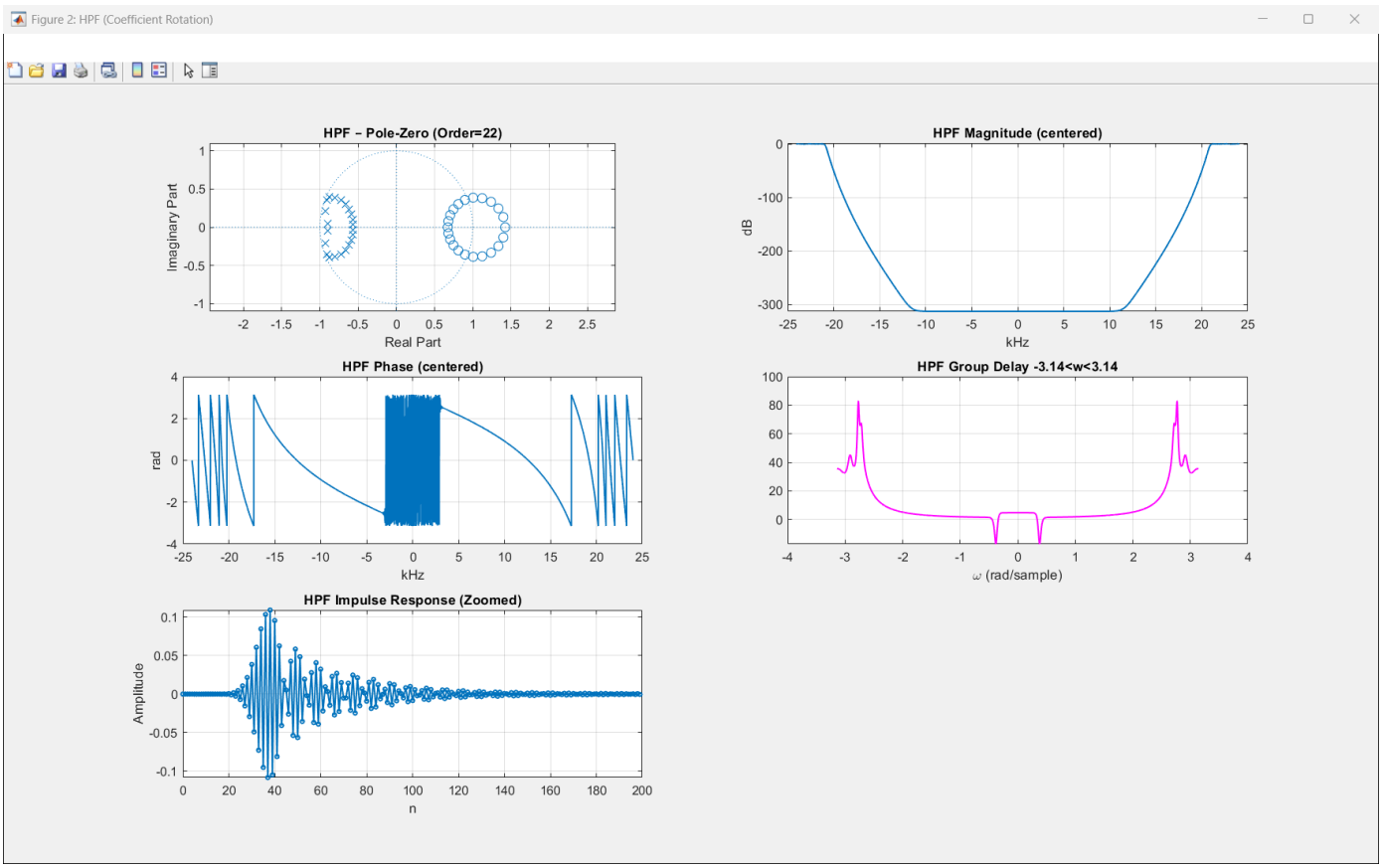


As designed before the pass band frequency is 3k and stop band is 4k so the gain is -50 dB in stop band.

LPF → HPF Transformation

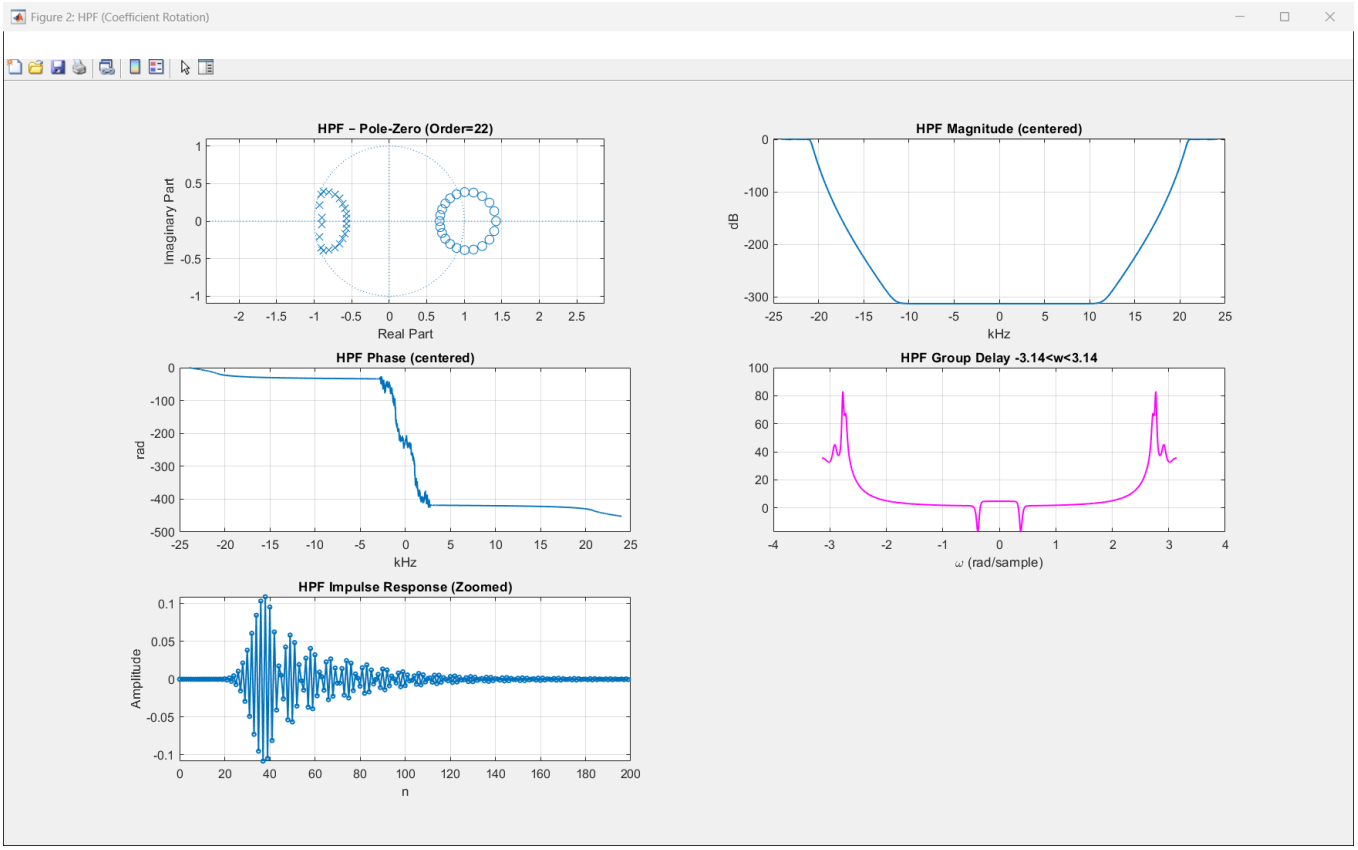
We rotated both zeros and poles by pi so when we start to rotate counter clock wise from x-axis we see the zeros first "low gain" and then at pi we see the poles "high gain".

This is the plots using normal phase



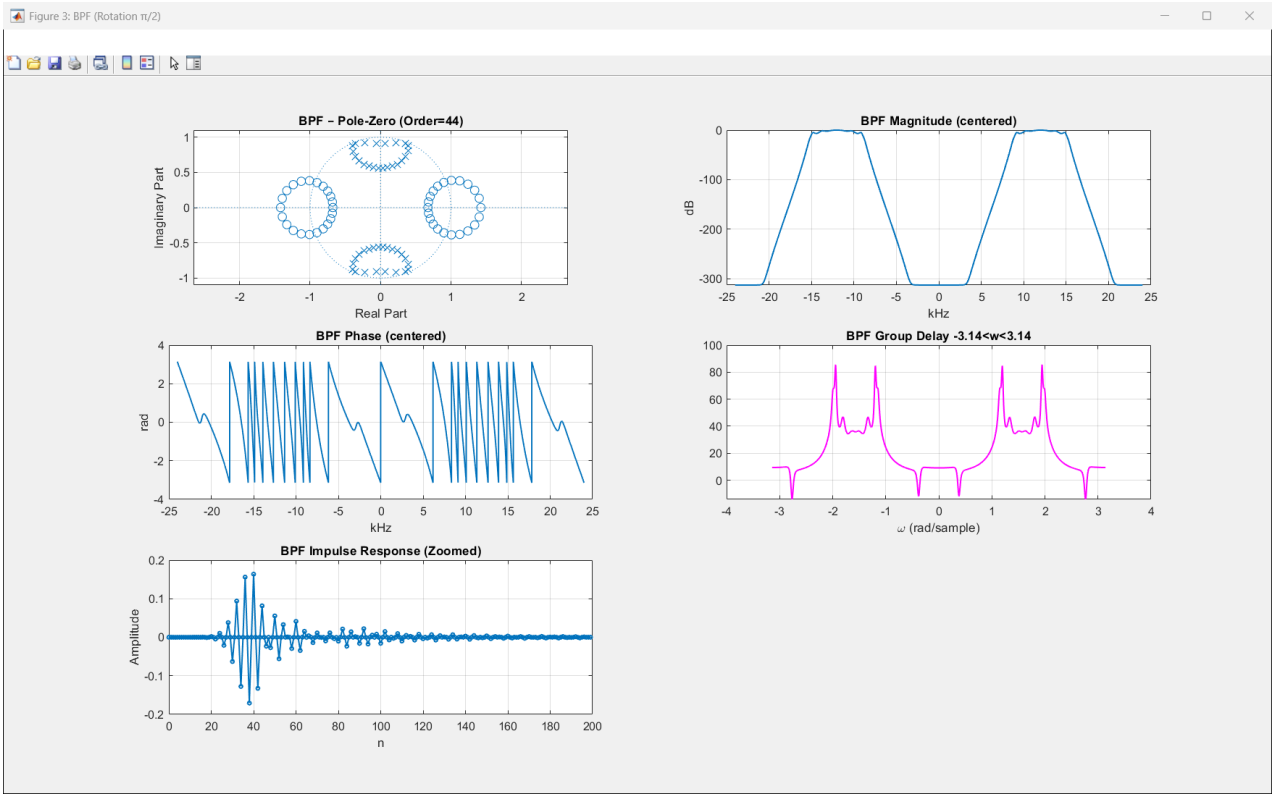
The magnitude response have high gain at high frequency as expected from a high pass filter. And the zigzag in phase response because we plot the normal phase but we will use the unwrap command see next figure.

If we used unwrap command again

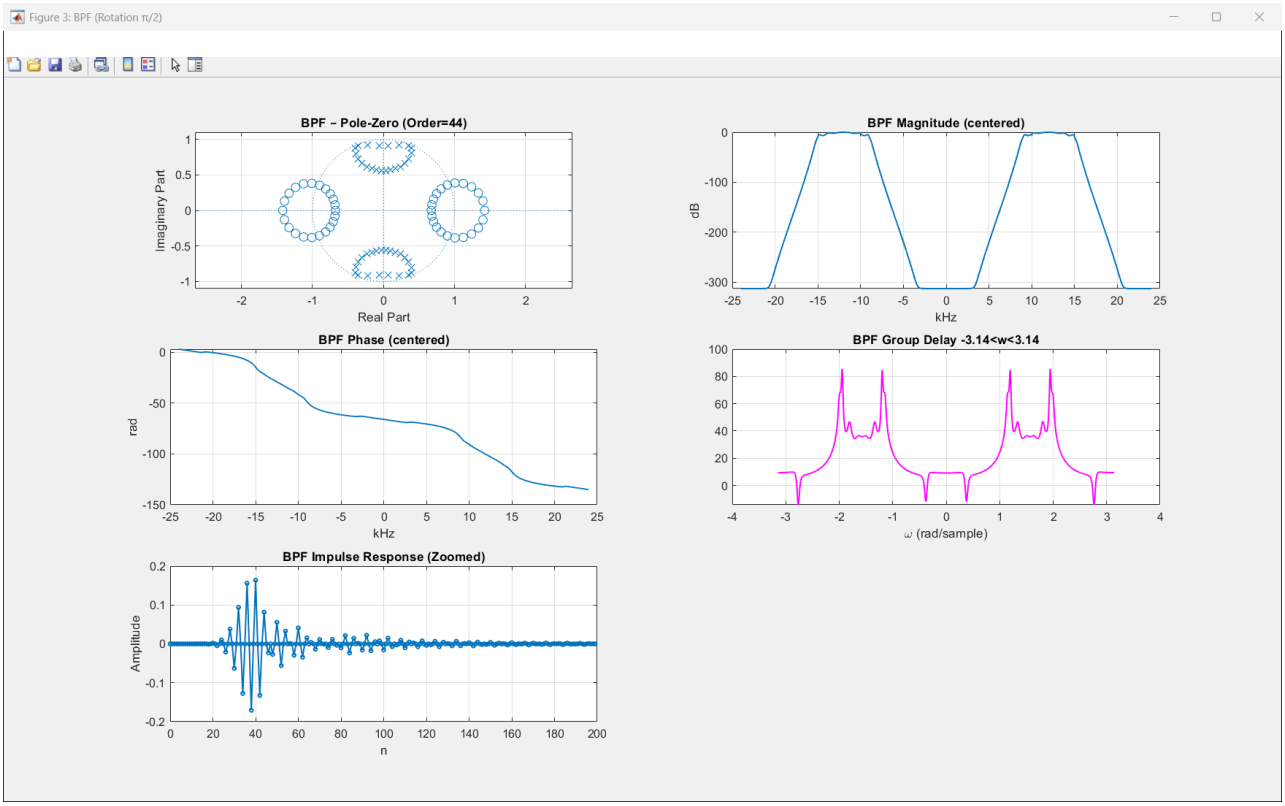


LPF → BPF Transformation

we remained the left zeros as it was and made another zeros from it and rotated it by -pi, and rotated the poles once by pi/2 and another by –pi/2.



As we done before use unwrap command



What to do to make sure that the filter coefficient is real ?

Real-valued filter coefficients are ensured by placing complex poles and zeros in conjugate pairs and eliminating numerical imaginary residues after transformations.

The symmetry around zero frequency confirms that the filters have real coefficients.

Task 3 codes

```
%% ============================================================
%  Butterworth Filter Transformations
%  LPF → HPF (Coefficient Rotation)
%  LPF → BPF (Rotation by ±π/2 in Z-plane)
% ============================================================

clear; clc; close all;

%% ------------------------------------------------------------
% Load audio
%% ------------------------------------------------------------
[x, Fs] = audioread("sample_audio_file.wav");
x = x(:,1);
x = x(:);

fprintf("Audio loaded: Fs = %d Hz\n", Fs);

Nfft = 16384*2;
omega = linspace(-pi, pi, Nfft);   % digital frequency axis (rad/sample)
f_center = (-Nfft/2:Nfft/2-1) * (Fs/Nfft);

%% ------------------------------------------------------------
% LPF Specifications (Butterworth ONLY)
%% ------------------------------------------------------------
fp  = 3000;
fsb = 4000;
Ap  = 1;
As  = 50;

Wp = fp/(Fs/2);
Ws = fsb/(Fs/2);

%% ------------------------------------------------------------
% Butterworth LPF (BASE FILTER – UNCHANGED)
%% ------------------------------------------------------------
[Nb, Wnb] = buttord(Wp, Ws, Ap, As);
[b_lpf, a_lpf] = butter(Nb, Wnb, 'low');

[z_lpf, p_lpf, k_lpf] = tf2zpk(b_lpf, a_lpf);
```

```
[Hwhole_lpf, ~] = freqz(b_lpf, a_lpf, Nfft, 'whole');
Hc_lpf = fftshift(Hwhole_lpf);

%% ------------------------------------------------------------
% LPF Plots
%% ------------------------------------------------------------
figure('Name','LPF','Position',[100 50 1400 900]);

subplot(2,1,1);
zplane(b_lpf, a_lpf);
grid on;
title(sprintf("LPF – Pole-Zero (Order=%d)", Nb));

subplot(2,1,2);
plot(f_center/1e3, 20*log10(abs(Hc_lpf)+eps),'LineWidth',1.2);
grid on;
xlabel("kHz"); ylabel("dB");
title("LPF Magnitude (centered)");

%% ============================================================
% LPF to HPF Conversion (Coefficient Rotation)
%% ============================================================
n_b = 0:length(b_lpf)-1;
n_a = 0:length(a_lpf)-1;

b_hpf = b_lpf .* ((-1).^n_b);
a_hpf = a_lpf .* ((-1).^n_a);

[Hwhole_hpf, ~] = freqz(b_hpf, a_hpf, Nfft, 'whole');
Hc_hpf = fftshift(Hwhole_hpf);
gd_hpf = fftshift(grpdelay(b_hpf, a_hpf, Nfft, 'whole'));

[h_imp_hpf, n_imp] = impz(b_hpf, a_hpf, 256);

y_hpf = filter(b_hpf, a_hpf, x);
MSE_hpf = mean((x - y_hpf).^2);

%% ------------------------------------------------------------
% HPF Plots
%% ------------------------------------------------------------
figure('Name','HPF (Coefficient Rotation)','Position',[100 50 1400 900]);

subplot(3,2,1);
zplane(b_hpf, a_hpf);
grid on;
title(sprintf("HPF – Pole-Zero (Order=%d)", Nb));

subplot(3,2,2);
plot(f_center/1e3,20*log10(abs(Hc_hpf)+eps),'LineWidth',1.2);
grid on;
xlabel("kHz"); ylabel("dB");
title("HPF Magnitude (centered)");

subplot(3,2,3);
plot(f_center/1e3,unwrap(angle(Hc_hpf)),'LineWidth',1.1);
```

```
grid on;
xlabel("kHz"); ylabel("rad");
title("HPF Phase (centered)");

subplot(3,2,4);
plot(omega,gd_hpf,'m','LineWidth',1.2);
grid on;
xlabel('\omega (rad/sample)');
title("HPF Group Delay -3.14<w<3.14");

subplot(3,2,5);
Nshow = 200;
plot(n_imp(1:Nshow),h_imp_hpf(1:Nshow),'o-','LineWidth',1.2,'MarkerSize',3);
grid on;
xlabel("n"); ylabel("Amplitude");
title("HPF Impulse Response (Zoomed)");

fprintf("HPF Order = %d | MSE = %.4e\n", Nb, MSE_hpf);

%% ============================================================
% LPF to BPF Conversion (Rotation by ±π/2)
%% ============================================================
phi = pi/2;

z1 = z_lpf;     % zeros remained
p1 = p_lpf * exp( 1j*phi); % poles rotated by pi/2

z2 = z_lpf * exp(-2j*phi); % zeros rotated by -pi
p2 = p_lpf * exp(-1j*phi); % poles rotated by -pi/2

z_bpf = [z1 ; z2];
p_bpf = [p1 ; p2];

[b_bpf,a_bpf] = zp2tf(z_bpf,p_bpf,1);

Hnorm = polyval(b_bpf, 1j) / polyval(a_bpf, 1j);
k_bpf = 1 / abs(Hnorm);
[b_bpf,a_bpf] = zp2tf(z_bpf,p_bpf,k_bpf);

b_bpf = real(b_bpf);
a_bpf = real(a_bpf);

[Hwhole_bpf, ~] = freqz(b_bpf, a_bpf, Nfft, 'whole');
Hc_bpf = fftshift(Hwhole_bpf);
gd_bpf = fftshift(grpdelay(b_bpf, a_bpf, Nfft, 'whole'));

[h_imp_bpf, n_imp] = impz(b_bpf, a_bpf, 256);

y_bpf = filter(b_bpf, a_bpf, x);
MSE_bpf = mean((x - y_bpf).^2);

%% ------------------------------------------------------------
% BPF Plots
%% ------------------------------------------------------------
figure('Name','BPF (Rotation π/2)','Position',[100 50 1400 900]);
```

```
subplot(3,2,1);
zplane(b_bpf, a_bpf);
grid on;
title(sprintf("BPF – Pole-Zero (Order=%d)", length(a_bpf)-1));

subplot(3,2,2);
plot(f_center/1e3,20*log10(abs(Hc_bpf)+eps),'LineWidth',1.2);
grid on;
xlabel("kHz"); ylabel("dB");
title("BPF Magnitude (centered)");

subplot(3,2,3);
plot(f_center/1e3,unwrap(angle(Hc_bpf)),'LineWidth',1.1);
grid on;
xlabel("kHz"); ylabel("rad");
title("BPF Phase (centered)");

subplot(3,2,4);
plot(omega,gd_bpf,'m','LineWidth',1.2);
grid on;
xlabel('\omega (rad/sample)');
title("BPF Group Delay -3.14<w<3.14");

subplot(3,2,5);
plot(n_imp(1:Nshow),h_imp_bpf(1:Nshow),'o-','LineWidth',1.2,'MarkerSize',3);
grid on;
xlabel("n"); ylabel("Amplitude");
title("BPF Impulse Response (Zoomed)");

fprintf("BPF Order = %d | MSE = %.4e\n", length(a_bpf)-1, MSE_bpf);
```

Thank

You

Instructors

Dr. Michael Ibrahim

Eng. Ahmed Khaled